



# RAPPORT DE PROJET

Master Informatique - Bases de données avancées

---

## TP 3 : Déduplication

*Détection des doublons et des similaires : algorithmes de similarité, MATCH et MERGE*

---

**Réalisé par :** Mathura SANTHALINGAM  
Vidur PARAKARAN  
Tesnime BEN SALEM  
Hassan CHARAF  
Amine BENAMARA  
Omar CHARKI

**Encadrant :** M. BOUFARÈS

## 1. Introduction

L'objectif de ce TP est de nettoyer une base de données : détecter les anomalies et surtout, éliminer les doubles exacts et les similaires.

## 2. Étude des algorithmes de similarité

### 2.1 Ce que nous avons fait

Nous avons travaillé dans la section MFB07 du script, avec la table matchval qui contient 31 couples de valeurs (valeur1, valeur2) répartis en quatre catégories : prénoms (FIRSTNAME), noms complets (FIRSTLASTNAME), villes (CITY) et adresses e-mail (EMAIL).

Ce que nous avons : la table matchval (31 lignes) ; chaque couple illustre un type d'écart : casse, faute de frappe, lettre ajoutée ou retirée, séparateur, accent, homophone, traduction.

Ce que nous avons réalisés :

- Exécution de la requête de comparaison qui calcule, pour chaque couple et après passage en majuscules (UPPER), cinq mesures : ED, EDS, JW, JWS, puis Soundex (SON1, SON2) et le Jaro-Winkler appliqué aux codes Soundex (S1S2).
- Premier constat : les colonnes Q-GRAM et MÉTAPHONE n'étaient pas encore calculées et affichaient un texte fixe. Nous avons donc écrit la fonction PL/SQL q\_gram\_similarity pour compléter la colonne Q-GRAM.

### 2.2 Les algorithmes

| Algorithme              | Principe                                                                                                                                              |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Edit distance (ED, EDS) | Nombre minimal d'insertions, suppressions ou substitutions pour passer d'une chaîne à l'autre. EDS ramène ce nombre à un score de 0 à 100.            |
| Jaro-Winkler (JW, JWS)  | Compte les caractères communs et les transpositions, avec un bonus quand le début des chaînes est identique. JW est entre 0 et 1, JWS entre 0 et 100. |
| Q-gram                  | Découpe chaque chaîne en sous-chaînes de q caractères (q = 2 ici) et mesure la proportion de sous-chaînes communes.                                   |
| Soundex (SON1, SON2)    | Code phonétique : première lettre + 3 chiffres. Deux mots qui se prononcent de la même façon doivent avoir le même code.                              |
| Métaphone               | Autre code phonétique plus précis que Soundex - on ne l'a pas défini                                                                                  |

### 2.3 Première exécution

La figure 1 montre la requête de comparaison telle qu'elle était écrite et son résultat. La requête sélectionne les colonnes de matchval puis les mesures ED, EDS, JW, JWS, SON1, SON2 et S1S2. Les emplacements Q-GRAM et MÉTAPHONE étaient encore en commentaire ou sous forme de texte fixe.

```
136 SELECT Idval,
137        categorieval,
138        valeur1,
139        valeur2,
140        -- S'écrit comme
141        UTL_MATCH.edit_distance(UPPER(valeur1), UPPER(valeur2)) ED,
142        UTL_MATCH.edit_distance_similarity(UPPER(valeur1), UPPER(valeur2)) EDS,
143        UTL_MATCH.jaro_winkler(UPPER(valeur1), UPPER(valeur2)) JW,
144        UTL_MATCH.jaro_winkler_similarity(UPPER(valeur1), UPPER(valeur2)) JWS,
145        -- 'Q-GRAM ???',
146        -- Se prononce comme
147        SOUNDEX(UPPER(valeur1)) SON1, SOUNDEX(UPPER(valeur2)) SON2,
148        UTL_MATCH.jaro_winkler_similarity(SOUNDEX(UPPER(valeur1)), SOUNDEX(UPPER(valeur2))) S1S2,
149        'METAPHONE ???'
150 FROM   matchval
151 -- on peut ajouter un seuil pour n'afficher que les tuples les plus proches
152 ORDER BY Idval;
153 /*
154 IDVAL    CATEGORIEVAL    VALEUR1    VALEUR2    ED    EDS    JW    JWS    'Q-GRAM???' SON1 SON2    S1S2    'METAPHONE???' 
```

Problems

Output

Debug Console

Terminal

Query Result

Task Monitor

SQL History

All rows fetched: 31 in 0.367 seconds

| IDVAL      | CATEGORIEVAL  | VALEUR1       | VALEUR2       | ED | EDS | JW                 | JWS | 'Q-GRAM??' | SON1 | SON2 | S1S2          | 'METAPHONE??' |
|------------|---------------|---------------|---------------|----|-----|--------------------|-----|------------|------|------|---------------|---------------|
| 5 A00-005  | FIRSTNAME     | Marie-Noël    | Marie Noël    | 1  | 90  | 0.96               | 96  | M654       | M654 | 100  | METAPHONE ??? |               |
| 6 A00-006  | FIRSTNAME     | Franc         | Frank         | 1  | 80  | 0.92               | 92  | F652       | F652 | 100  | METAPHONE ??? |               |
| 7 A00-007  | FIRSTNAME     | Mbarak        | Moubarak      | 2  | 75  | 0.9249999999999999 | 92  | M162       | M162 | 100  | METAPHONE ??? |               |
| 8 A00-008  | FIRSTNAME     | Inès          | Ines          | 2  | 60  | 0.8266666666666667 | 82  | I520       | I520 | 100  | METAPHONE ??? |               |
| 9 A00-009  | FIRSTNAME     | Inès          | Iness         | 2  | 60  | 0.7866666666666667 | 78  | I520       | I520 | 100  | METAPHONE ??? |               |
| 10 A00-010 | FIRSTNAME     | Inès          | Yneès         | 2  | 67  | 0.8222222222222222 | 82  | I520       | Y520 | 83   | METAPHONE ??? |               |
| 11 A00-011 | FIRSTNAME     | Inès          | Agnès         | 2  | 67  | 0.8222222222222222 | 82  | I520       | A252 | 50   | METAPHONE ??? |               |
| 12 A00-021 | FIRSTLASTNAME | Peter Parker  | Pete Parker   | 1  | 92  | 0.9287878787878787 | 92  | P361       | P316 | 93   | METAPHONE ??? |               |
| 13 A00-022 | FIRSTLASTNAME | Peter Parker  | peter parker  | 0  | 100 | 1                  | 100 | P361       | P361 | 100  | METAPHONE ??? |               |
| 14 A00-023 | FIRSTLASTNAME | Clark Kent    | Claire Kent   | 2  | 82  | 0.9083164983164983 | 90  | C462       | C462 | 100  | METAPHONE ??? |               |
| 15 A00-024 | FIRSTLASTNAME | Wonder Woman  | Ponder Woman  | 1  | 92  | 0.9444444444444444 | 94  | W536       | P536 | 83   | METAPHONE ??? |               |
| 16 A00-025 | FIRSTLASTNAME | Superman      | Superman      | 0  | 100 | 1                  | 100 | S165       | S165 | 100  | METAPHONE ??? |               |
| 17 A00-026 | FIRSTLASTNAME | The Hulk      | Iron Man      | 8  | 0   | 0.4166666666666667 | 41  | T420       | I655 | 0    | METAPHONE ??? |               |
| 18 A00-027 | FIRSTLASTNAME | Harissa FORD  | Harrison Ford | 2  | 85  | 0.9343822843822843 | 93  | H621       | H625 | 88   | METAPHONE ??? |               |
| 19 A00-028 | FIRSTLASTNAME | Bus WILLY     | Bruce Willy   | 3  | 73  | 0.8848484848484848 | 88  | B240       | B624 | 85   | METAPHONE ??? |               |
| 20 A00-030 | FIRSTLASTNAME | Brittne Bardo | Brittne Bardo | 1  | 92  | 0.9714785714285714 | 97  | B673       | B673 | 100  | METAPHONE ??? |               |

Figure 1 – Requête de comparaison et résultat (lignes 5 à 19), avant l'ajout de la fonction Q-gram

## 2.4 Observations tirées des captures

- **Casse.** Adam/ADAM et Peter Parker/peter parker obtiennent ED = 0 et JWS = 100. Le passage en majuscules avant comparaison efface complètement la différence de casse.
- **Faute de frappe.** Adam/Adem : ED = 1, EDS = 75, JWS = 86. Adam/Adams : EDS = 80, JWS = 96. Jaro-Winkler est plus tolérant que l'edit distance, car il valorise le préfixe commun.
- **Séparateur.** Marie-Noel/Marie Noël : ED = 1, EDS = 90 et JWS = 96, la différence est minime.
- **Phonétique, points forts.** Franc/Frank, Rahma/Rama, Mbarak/Moubarak et Inès/Ines/Iness ont des codes Soundex identiques (F652, R500, M162, I520) donc S1S2 = 100, alors que leur orthographe diffère.
- **Phonétique, limite.** Soundex conserve la première lettre. Peter/Pete donne P361 contre P316 (S1S2 = 93), Wonder/Ponder donne W536 contre P536 (83), Inès/Yneès donne I520 contre Y520 (83) et Inès/Agnès donne I520 contre A252 (50).
- **Accents.** Inès/Ines a ED = 2 et EDS = 60 dans notre capture, alors que le commentaire du script indique ED = 1 et EDS = 75. Une hypothèse est un comptage en octets (è occupe 2 octets en UTF-8).
- **Couples très différents.** The Hulk/Iron Man : ED = 8, EDS = 0, JWS = 41, correctement rejeté par les deux mesures.

## 2.5 Fonction Q-gram

La fonction `q_gram_similarity(p_str1, p_str2, p_q)` implémente l'algorithme des q-grammes (avec  $q = 2$  par défaut, soit des bigrammes) pour évaluer la similarité entre deux chaînes en s'appuyant sur le coefficient de Sørensen-Dice.

Après avoir écarté les cas particuliers (valeurs nulles, chaînes plus courtes que  $q$ , ou chaînes strictement identiques qui retournent directement 100 %), on déconstruit la première chaîne en sous-séquences de 2 caractères. Pour chaque bigramme extrait, elle vérifie son existence dans la seconde chaîne.

Lorsqu'une correspondance est trouvée, le compteur d'intersections est incrémenté, et **seule la première lettre** de l'occurrence dans la seconde chaîne est remplacée par un caractère masqué (#). Cette méthode empêche de compter un même bigramme plusieurs fois tout en préservant les lettres

suivantes, ce qui est indispensable pour détecter les bigrammes qui se chevauchent naturellement. Le score final est ensuite par :  $(2 \times \text{intersections}) / (\text{total des bigrammes de la chaîne 1} + \text{total des bigrammes de la chaîne 2}) \times 100$

#### Exemple d'exécution : Comparaison entre "RAHMA" et "RAMA"

- **Chaîne 1 ("RAHMA")** : Contient 4 bigrammes (RA, AH, HM, MA).
- **Chaîne 2 ("RAMA")** : Contient 3 bigrammes (RA, AM, MA).

#### Déroulement de l'algorithme :

1. Le 1er bigramme de la chaîne 1 (RA) est cherché dans "RAMA". Il est trouvé. Le compteur passe à 1. La chaîne 2 est masquée et devient #AMA.
2. Le 2ème bigramme (AH) est cherché dans #AMA. Introuvable. Le compteur reste à 1.
3. Le 3ème bigramme (HM) est cherché dans #AMA. Introuvable. Le compteur reste à 1.
4. Le 4ème bigramme (MA) est cherché dans #AMA. Il est trouvé. Le compteur passe à 2. La chaîne 2 est masquée et devient #A#A.

Le calcul final donne :  $(2 \times 2 \text{ intersections}) / (4 + 3 \text{ bigrammes}) \times 100 = 4 / 7 \times 100$ . La fonction retourne un score de similarité de **57 %**.

```

181 SELECT Idval,
182        categorieval,
183        valeur1,
184        valeur2,
185        -- S'écrit comme
186        UTL_MATCH.edit_distance(UPPER(valeur1), UPPER(valeur2)) ED,
187        UTL_MATCH.edit_distance_similarity(UPPER(valeur1), UPPER(valeur2)) EDS,
188        UTL_MATCH.jaro_winkler(UPPER(valeur1), UPPER(valeur2)) JW,
189        UTL_MATCH.jaro_winkler_similarity(UPPER(valeur1), UPPER(valeur2)) JWS,
190        q_gram_similarity(UPPER(valeur1), UPPER(valeur2)) Q_GRAM,
191        -- Se prononce comme
192        SOUNDEX(UPPER(valeur1)) SON1, SOUNDEX(UPPER(valeur2)) SON2,
193        UTL_MATCH.jaro_winkler_similarity(SOUNDEX(UPPER(valeur1)), SOUNDEX(UPPER(valeur2))) S1S2,
194        'METAPHONE ???'
195 FROM   matchval
196 -- on peut ajouter un seuil pour n'afficher que les tuples les plus proches
197 ORDER BY Idval;
198

```

| IDVAL | CATEGORIEVAL | VALEUR1       | VALEUR2        | ED | EDS | JW                 | JWS | Q_GRAM | SON1 | SON2 | S1S2 | 'METAPHONE???' |
|-------|--------------|---------------|----------------|----|-----|--------------------|-----|--------|------|------|------|----------------|
| 1     | A00-001      | FIRSTNAME     | Adam           | 0  | 100 | 1                  | 100 | 100    | A350 | A350 | 100  | METAPHONE ???  |
| 2     | A00-002      | FIRSTNAME     | Adam           | 1  | 75  | 0.8666666666666666 | 86  | 33     | A350 | A350 | 100  | METAPHONE ???  |
| 3     | A00-003      | FIRSTNAME     | Adam           | 1  | 80  | 0.96               | 96  | 86     | A350 | A352 | 88   | METAPHONE ???  |
| 4     | A00-004      | FIRSTNAME     | Rahma          | 1  | 80  | 0.9466666666666665 | 94  | 57     | R500 | R500 | 100  | METAPHONE ???  |
| 5     | A00-005      | FIRSTNAME     | Marie-Noel     | 1  | 90  | 0.96               | 96  | 78     | M654 | M654 | 100  | METAPHONE ???  |
| 6     | A00-006      | FIRSTNAME     | Frank          | 1  | 80  | 0.92               | 92  | 75     | F652 | F652 | 100  | METAPHONE ???  |
| 7     | A00-007      | FIRSTNAME     | Mbarak         | 2  | 75  | 0.9249999999999999 | 92  | 67     | M162 | M162 | 100  | METAPHONE ???  |
| 8     | A00-008      | FIRSTNAME     | Ines           | 2  | 60  | 0.8266666666666667 | 82  | 33     | I520 | I520 | 100  | METAPHONE ???  |
| 9     | A00-009      | FIRSTNAME     | Ines           | 2  | 60  | 0.7866666666666667 | 78  | 29     | I520 | I520 | 100  | METAPHONE ???  |
| 10    | A00-010      | FIRSTNAME     | Ines           | 2  | 67  | 0.8222222222222222 | 82  | 29     | I520 | I520 | 83   | METAPHONE ???  |
| 11    | A00-011      | FIRSTNAME     | Ines           | 2  | 67  | 0.8222222222222222 | 82  | 57     | I520 | A252 | 50   | METAPHONE ???  |
| 12    | A00-021      | FIRSTLASTNAME | Peter Parker   | 1  | 92  | 0.9287878787878787 | 92  | 76     | P361 | P316 | 93   | METAPHONE ???  |
| 13    | A00-022      | FIRSTLASTNAME | Peter Parker   | 0  | 100 | 1                  | 100 | 100    | P361 | P361 | 100  | METAPHONE ???  |
| 14    | A00-023      | FIRSTLASTNAME | Clark Kent     | 2  | 82  | 0.9083164983164983 | 90  | 63     | C462 | C462 | 100  | METAPHONE ???  |
| 15    | A00-024      | FIRSTLASTNAME | Wonder Woman   | 1  | 92  | 0.9444444444444444 | 94  | 82     | W536 | P536 | 83   | METAPHONE ???  |
| 16    | A00-025      | FIRSTLASTNAME | Superman       | 0  | 100 | 1                  | 100 | 100    | S165 | S165 | 100  | METAPHONE ???  |
| 17    | A00-026      | FIRSTLASTNAME | The Hulk       | 8  | 0   | 0.4166666666666667 | 41  | 0      | T420 | I655 | 0    | METAPHONE ???  |
| 18    | A00-027      | FIRSTLASTNAME | Harissa FORD   | 2  | 85  | 0.9343822843822843 | 93  | 78     | H621 | H625 | 88   | METAPHONE ???  |
| 19    | A00-028      | FIRSTLASTNAME | Bus WILLY      | 3  | 73  | 0.8848484848484848 | 88  | 56     | B240 | B624 | 85   | METAPHONE ???  |
| 20    | A00-029      | FIRSTLASTNAME | Brigitte Bardo | 1  | 93  | 0.9714285714285714 | 97  | 85     | B623 | B623 | 100  | METAPHONE ???  |

**Figure 2** – Requête de comparaison et résultat (lignes 5 à 19) après l'ajout de la fonction Q-gram

N.B. : Nous avons privilégié les bigrammes (q=2) plutôt que les 5-grammes car sur des chaînes courtes (ex: "Adam", "Pari"), un 5-gramme ne génère aucune sous-chaîne valide, empêchant toute comparaison. De plus, la moindre faute de frappe détruit instantanément un bloc entier de 5 lettres. À l'inverse, le découpage en bigrammes offre un maillage fin et tolérant : il absorbe parfaitement les erreurs de saisie, les lettres omises et les inversions de mots.

### 3. Élimination des doubles et des similaires

La procédure ELIMINEDOUBSIMIL(NOMTAB, AttributsClé, ListeTousAttributs, SEUIL1, SEUIL2) sauvegarde la table source dans S, construit une clé de blocage NEWKEY en concaténant les attributs, compare

toutes les paires par produit cartésien (EDS et JWS), puis retient comme similaires les paires dont les deux scores dépassent les seuils et dont les 3 premiers caractères sont identiques. Pour chaque couple retenu, elle indique la clé à garder, c'est-à-dire la plus longue.

## 4. Stratégie de détection (MATCH) et de fusion (MERGE)

Deux valeurs très proches peuvent désigner des personnes différentes, et deux valeurs très éloignées peuvent désigner la même. Cette section décrit la réflexion qui guide notre nettoyage : comment choisir l'algorithme selon le type de donnée, comment la sémantique et les autres colonnes permettent de trancher, et que faire d'un groupe de lignes reconnues comme identiques.

### 4.1 Pourquoi comparer une seule colonne ne suffit pas

- **Très similaire, mais distinct.** fb@lipn... et yb@lipn... ont un EDS de 96, mais ce sont deux adresses différentes. Clark Kent et Claire Kent ont le même code Soundex (C462) alors que le prénom diffère. Inès et Agnès atteignent un JWS de 82 alors que ce sont deux prénoms distincts.
- **Très différent, mais même entité.** Pékin / Beijing (EDS 29) et Londres / London (EDS 58) sont les mêmes villes écrites dans deux langues. Les distances de chaînes les rejettent à tort.

Pour être sûr qu'il ne s'agit pas de la même personne (ou qu'il s'agit bien d'elle), il faut des preuves convergentes sur d'autres colonnes :

| Situation observée                                                       | Décision                                         |
|--------------------------------------------------------------------------|--------------------------------------------------|
| Nom similaire + même date de naissance, ou même email, ou même téléphone | Même personne : doublon                          |
| Nom similaire + date de naissance différente                             | Personnes différentes (cas Traifort Eve / Nadia) |
| Nom similaire + autre colonne NULL ou non comparable                     | À vérifier : aucune fusion automatique           |
| Nom identique + tout le reste différent                                  | Homonymes : on ne fusionne pas                   |

### 4.2 Réflexion sur le choix des colonnes d'appariement (Matching Keys)

Comparer l'intégralité des colonnes de deux tables "à l'aveugle" serait très coûteux en ressources de calcul, et introduirait aussi un "bruit" statistique qui pourrait fausser les résultats.

Le choix des colonnes à confronter ne se fait pas au hasard. On **propose** qu'il repose sur quatre critères :

1. **La qualité et la complétude des colonnes** : Une colonne, même très discriminante, est inutile pour la comparaison si elle est vide à 90 %. Avant d'inclure une colonne dans notre clé de blocage ou notre calcul de similarité, nous évaluons son taux de remplissage via les métriques M100 (Number of MISSING values) et M101 (Number of NON-MISSING values). Nous excluons volontairement les colonnes trop éparses (sparse data) de l'équation de Matching, car l'absence de donnée d'un côté fausserait le calcul de distance de l'autre, empêchant la détection légitime d'un doublon.

2. **Le pouvoir discriminant de la donnée (Unicité)** : Une colonne GENRECLI ou CATCLI possède un pouvoir discriminant très faible : des milliers de lignes partagent la même valeur. À l'inverse, des colonnes comme MAILCLI ou TELCLI sont des identifiants quasi-unique. La stratégie consiste à sélectionner des colonnes discriminantes

3. **L'alignement sémantique (Sens de la donnée)** : Pour fusionner des données issues de tables différentes (comme ce fut le cas pour les tables ETUDIUTV et ETUDIG), il faut s'assurer que l'on compare des éléments sémantiquement identiques.

- Par le nommage : On s'appuie d'abord sur la métadonnée explicite (ex: VILLEE correspond à VILLEETUD).
- Par le dictionnaire de structure : L'utilisation de notre table de métadonnées META04\_DD\_DATASTRUCTURES1 permet de vérifier que les colonnes ciblées partagent le même rôle métier dans le système d'information.

4. **La compatibilité syntaxique et statistique (Data Profiling)** : Grâce au profilage stocké dans META02\_DD\_MESURES, nous pouvons identifier des paires de colonnes candidates à la comparaison en analysant leur morphologie :

- **Types et sous-types** : On ne compare que des colonnes partageant le même DOMINANT SYNTAX TYPE (M120). Comparer une chaîne alphabétique à une colonne strictement numérique n'a aucun sens algorithmique.
- **Distributions** : Si deux colonnes de tables différentes partagent les mêmes longueurs moyennes (M160) et les mêmes valeurs minimales et maximales (M161, M162), il y a une forte probabilité qu'elles décrivent la même réalité métier et qu'elles soient pertinentes pour un appariement.

## 4.2 Choix de l'algorithme selon le type de donnée

Pour choisir l'algorithme adapté à chaque type de donnée, nous avons exécuté les quatre mesures (Edit\_Dist\_Sim, Jaro\_Winkler\_Sim, Q\_Gram\_Sim et Phonetique\_Sim) séparément sur chaque catégorie de la table matchval : prénoms (figure 5), noms complets (figure 6), villes (figure 7) et adresses e-mail (figure 8). La colonne Phonetique\_Sim est le Jaro-Winkler appliqué aux codes Soundex : la valeur 100 signifie que les deux codes sont identiques.

```

200 -- Test prénom
201
202 SELECT Idval, valeur1, valeur2,
203        UTL_MATCH.edit_distance_similarity(UPPER(valeur1), UPPER(valeur2)) AS Edit_Dist_Sim,
204        UTL_MATCH.jaro_winkler_similarity(UPPER(valeur1), UPPER(valeur2)) AS Jaro_Winkler_Sim,
205        q_gram_similarity(UPPER(valeur1), UPPER(valeur2)) AS Q_Gram_Sim,
206        UTL_MATCH.jaro_winkler_similarity(SOUNDEX(UPPER(valeur1)), SOUNDEX(UPPER(valeur2))) AS Phonetique_Sim
207 FROM matchval
208 WHERE categorieval = 'FIRSTNAME'
209 ORDER BY Idval;
210

```

Problems Output Debug Console Terminal **Query Result** Task Monitor Script Output SQL History

All rows fetched: 11 in 0.070 seconds

|    | IDVAL   | VALEUR1    | VALEUR2    | EDIT_DIST_SIM | JARO_WINKLER_SIM | Q_GRAM_SIM | PHONETIQUE_SIM |
|----|---------|------------|------------|---------------|------------------|------------|----------------|
| 1  | A00-001 | Adam       | ADAM       | 100           | 100              | 100        | 100            |
| 2  | A00-002 | Adam       | Adem       | 75            | 86               | 33         | 100            |
| 3  | A00-003 | Adam       | Adams      | 80            | 96               | 86         | 88             |
| 4  | A00-004 | Rahma      | Rama       | 80            | 94               | 57         | 100            |
| 5  | A00-005 | Marie-Noel | Marie Noel | 90            | 96               | 78         | 100            |
| 6  | A00-006 | Franc      | Frank      | 80            | 92               | 75         | 100            |
| 7  | A00-007 | Mbarak     | Moubarak   | 75            | 92               | 67         | 100            |
| 8  | A00-008 | Inès       | Ines       | 60            | 82               | 33         | 100            |
| 9  | A00-009 | Inès       | Iness      | 60            | 78               | 29         | 100            |
| 10 | A00-010 | Inès       | Yneès      | 67            | 82               | 29         | 83             |
| 11 | A00-011 | Inès       | Agnès      | 67            | 82               | 57         | 50             |

Figure 3 – Test des algorithmes sur les prénoms (catégorie FIRSTNAME, 11 couples)

```

211 -- Test nom
212 SELECT Idval, valeur1, valeur2,
213        UTL_MATCH.edit_distance_similarity(UPPER(valeur1), UPPER(valeur2)) AS Edit_Dist_Sim,
214        UTL_MATCH.jaro_winkler_similarity(UPPER(valeur1), UPPER(valeur2)) AS Jaro_Winkler_Sim,
215        q_gram_similarity(UPPER(valeur1), UPPER(valeur2)) AS Q_Gram_Sim,
216        UTL_MATCH.jaro_winkler_similarity(SOUNDEX(UPPER(valeur1)), SOUNDEX(UPPER(valeur2))) AS Phonetique_Sim
217 FROM matchval
218 WHERE categorieval = 'FIRSTLASTNAME'
219 ORDER BY Idval;
220
221

```

Problems Output Debug Console Terminal **Query Result** Task Monitor Script Output SQL History

All rows fetched: 12 in 0.092 seconds

|    | IDVAL   | VALEUR1        | VALEUR2        | EDIT_DIST_SIM | JARO_WINKLER_SIM | Q_GRAM_SIM | PHONETIQUE_SIM |
|----|---------|----------------|----------------|---------------|------------------|------------|----------------|
| 1  | A00-021 | Peter Parker   | Pete Parker    | 92            | 92               | 76         | 93             |
| 2  | A00-022 | Peter Parker   | peter parker   | 100           | 100              | 100        | 100            |
| 3  | A00-023 | Clark Kent     | Claire Kent    | 82            | 90               | 63         | 100            |
| 4  | A00-024 | Wonder Woman   | Ponder Woman   | 92            | 94               | 82         | 83             |
| 5  | A00-025 | Superman       | Superman       | 100           | 100              | 100        | 100            |
| 6  | A00-026 | The Hulk       | Iron Man       | 0             | 41               | 0          | 0              |
| 7  | A00-027 | Harissa FORD   | Harrison Ford  | 85            | 93               | 78         | 88             |
| 8  | A00-028 | Bus WILLY      | Bruce Willy    | 73            | 88               | 56         | 85             |
| 9  | A00-029 | Brigitte Bardo | Brigitte Fardo | 93            | 97               | 85         | 100            |
| 10 | A00-030 | Hedi Mufti     | Eddy Murfi     | 50            | 80               | 33         | 66             |
| 11 | A00-031 | Alain DE LOIN  | Alain DELON    | 85            | 96               | 73         | 100            |
| 12 | A00-032 | De par de      | 2 par 2        | 56            | 75               | 57         | 50             |

**Figure 4** – Test des algorithmes sur les noms complets (catégorie FIRSTLASTNAME, 12 couples)

```

221 -- tests sur les villes
222
223 SELECT Idval, valeur1, valeur2,
224        UTL_MATCH.edit_distance_similarity(UPPER(valeur1), UPPER(valeur2)) AS Edit_Dist_Sim,
225        UTL_MATCH.jaro_winkler_similarity(UPPER(valeur1), UPPER(valeur2)) AS Jaro_Winkler_Sim,
226        q_gram_similarity(UPPER(valeur1), UPPER(valeur2)) AS Q_Gram_Sim,
227        UTL_MATCH.jaro_winkler_similarity(SOUNDEX(UPPER(valeur1)), SOUNDEX(UPPER(valeur2))) AS Phonetique_Sim
228 FROM matchval
229 WHERE categorieval = 'CITY'
230 ORDER BY Idval;
231

```

Problems Output Debug Console Terminal **Query Result** Task Monitor Script Output SQL History

All rows fetched: 5 in 0.125 seconds

|   | IDVAL   | VALEUR1 | VALEUR2 | EDIT_DIST_SIM | JARO_WINKLER_SIM | Q_GRAM_SIM | PHONETIQUE_SIM |
|---|---------|---------|---------|---------------|------------------|------------|----------------|
| 1 | A00-041 | Paris   | PARISS  | 72            | 94               | 80         | 100            |
| 2 | A00-042 | Paris   | Pari    | 80            | 96               | 86         | 86             |
| 3 | A00-043 | Pékin   | Beijing | 29            | 53               | 20         | 66             |
| 4 | A00-044 | Londres | Londre  | 86            | 97               | 91         | 100            |
| 5 | A00-045 | Londres | London  | 58            | 84               | 55         | 88             |

**Figure 5** – Test des algorithmes sur les villes (catégorie CITY, 5 couples)

```

232 -- tests sur les emails
233 SELECT Idval, valeur1, valeur2,
234        UTL_MATCH.edit_distance_similarity(UPPER(valeur1), UPPER(valeur2)) AS Edit_Dist_Sim,
235        UTL_MATCH.jaro_winkler_similarity(UPPER(valeur1), UPPER(valeur2)) AS Jaro_Winkler_Sim,
236        q_gram_similarity(UPPER(valeur1), UPPER(valeur2)) AS Q_Gram_Sim,
237        UTL_MATCH.jaro_winkler_similarity(SOUNDEX(UPPER(valeur1)), SOUNDEX(UPPER(valeur2))) AS Phonetique_Sim
238 FROM matchval
239 WHERE categorieval = 'EMAIL'
240 ORDER BY Idval;

```

Problems Output Debug Console Terminal **Query Result** Task Monitor Script Output SQL History

All rows fetched: 3 in 0.079 seconds

|   | IDVAL   | VALEUR1                 | VALEUR2                 | EDIT_DIST_SIM | JARO_WINKLER_SIM | Q_GRAM_SIM | PHONETIQUE_SIM |
|---|---------|-------------------------|-------------------------|---------------|------------------|------------|----------------|
| 1 | A00-061 | fb@lipn.univ-paris13.fr | fb@lipn.univ-paris13.fr | 100           | 100              | 100        | 100            |
| 2 | A00-062 | fb@lipn.univ-paris13.fr | yb@lipn.univ-paris13.fr | 96            | 97               | 95         | 50             |
| 3 | A00-063 | fb@lipn.univ-paris13.fr | fb@iutv.univ-paris13.fr | 83            | 91               | 77         | 85             |

**Figure 6** – Test des algorithmes sur les adresses e-mail (catégorie EMAIL, 3 couples)

Le tableau suivant résume l'algorithme retenu pour chaque type de donnée et la raison de ce choix.

| Type de donnée                        | Algorithme retenu                   | Justification                                                                                                                                                                                                                            |
|---------------------------------------|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>FIRSTNAME (Prénoms)</b>            | Jaro-Winkler / Phonétique (Soundex) | Jaro-Winkler est particulièrement adapté aux noms, prénoms et chaînes courtes car il donne plus de poids au début du mot (idéal pour les troncatures). La phonétique rattrape les erreurs de transcription orale (ex : Adem/Adam).       |
| <b>FIRSTLASTNAME (Noms + Prénoms)</b> | Q-Gram                              | L'approche par Q-gram (bigrammes) est insensible à l'ordre des mots. Elle excelle pour détecter des similarités quand le nom et le prénom sont inversés (ex : « Peter Parker » vs « Parker Peter ») ou quand les espaces sont mal gérés. |
| <b>CITY (Villes)</b>                  | Jaro-Winkler / Q-Gram               | Les noms de villes sont des chaînes courtes sujettes à des fautes de frappe en fin de mot (Pari/Paris, Londre/Londres). Jaro-Winkler absorbe très bien ces variations.                                                                   |
| <b>EMAIL (Identifiants, adresses)</b> | Edit Distance (Levenshtein)         | Une adresse email est une donnée stricte. Une différence s'explique souvent par une touche de clavier tapée à côté (ex : yb au lieu de fb). La phonétique ou l'inversion de mots n'ont aucune pertinence ici.                            |

### 4.3 Définition des seuils de similarité : une approche croisée

Lors de la phase de déduplication, une **erreur** commune consisterait à définir un **seuil** de similarité global (par exemple 80 %) et à l'appliquer à **l'ensemble de la base de données**. Notre réflexion nous a menés à la conclusion qu'un "bon" seuil n'existe pas dans l'absolu : il doit être défini par l'intersection de deux facteurs : **la colonne choisie et l'algorithme utilisé**.

1. **La colonne** : Notre tolérance à l'erreur doit varier selon ce que représente la donnée. Par exemple, une adresse email est un identifiant strict. Dans la table matchval, une seule touche de clavier d'écart entre fb@lipn.univ-paris13.fr et yb@lipn.univ-paris13.fr modifie totalement l'identité de l'individu. Sur ce type de champ, **la tolérance doit être quasi nulle**. À l'inverse, un champ combinant "Nom et Prénom" (FIRSTLASTNAME) subit fréquemment des inversions ("Peter Parker" vs "peter parker") ou des fautes de frappe qui nécessitent une plus grande flexibilité.

2. **L'algorithmique** : L'algorithme de **Jaro-Winkler** accorde un poids important aux préfixes partagés. Il va donc "pardonner" très facilement une erreur de fin de mot (ex: "Pari" vs "Paris") et générer des scores naturellement hauts. À l'inverse, une approche par **Q-Grams** (bigrammes) déconstruit la chaîne et sanctionne plus lourdement les petites variations. **Un score de 80 % en Jaro-Winkler ne traduit donc pas la même distance qu'un score de 80 % en Q-Gram**.

#### 3. Définition de notre stratégie de seuils :

- Identifiants stricts (Emails) → Edit Distance (Seuil > 95 %) : L'approche par distance d'édition est idéale pour capter la modification d'un seul caractère. Le seuil très haut garantit l'absence de faux positifs.
- Chaînes courtes et simples (Prénoms, Villes) → Jaro-Winkler (Seuil > 85 %) : Cet algorithme absorbe bien les variations orthographiques légères (ex: "Ines" vs "Inès") et les troncatures, fréquentes sur ces champs.
- Chaînes longues et composites (Noms complets) → Q-Gram (Seuil > 75 %) : En cherchant le taux de chevauchement des bigrammes indépendamment de l'ordre des mots, cet algorithme est

parfait pour détecter qu'un nom inversé ("Peter Parker" vs "Pete Parker") appartient à la même personne. Le seuil est volontairement abaissé car le calcul par bigrammes est plus sévère.

Tests pour les seuils :

```
241 SELECT categorieval, valeur1, valeur2,
242 | | | UTL_MATCH.edit_distance_similarity(UPPER(valeur1), UPPER(valeur2)) AS Score_Edit_Distance
243 FROM matchval
244 WHERE categorieval = 'EMAIL'
245 | AND UTL_MATCH.edit_distance_similarity(UPPER(valeur1), UPPER(valeur2)) > 95
246 ORDER BY Score_Edit_Distance DESC;
247
```

Problems Output Debug Console Terminal Query Result Task Monitor Script Output SQL History

All rows fetched: 2 in 0.056 seconds

|   | CATEGORIEVAL | VALEUR1                 | VALEUR2                 | SCORE_EDIT_DISTANCE |
|---|--------------|-------------------------|-------------------------|---------------------|
| 1 | EMAIL        | fb@lipn.univ-paris13.fr | fb@lipn.univ-paris13.fr | 100                 |
| 2 | EMAIL        | fb@lipn.univ-paris13.fr | yb@lipn.univ-paris13.fr | 96                  |

```
248 SELECT categorieval, valeur1, valeur2,
249 | | | UTL_MATCH.jaro_winkler_similarity(UPPER(valeur1), UPPER(valeur2)) AS Score_Jaro_Winkler
250 FROM matchval
251 WHERE categorieval IN ('FIRSTNAME', 'CITY')
252 | AND UTL_MATCH.jaro_winkler_similarity(UPPER(valeur1), UPPER(valeur2)) > 85
253 ORDER BY categorieval, Score_Jaro_Winkler DESC;
254
```

Problems Output Debug Console Terminal Query Result Task Monitor Script Output SQL History

All rows fetched: 10 in 0.071 seconds

|    | CATEGORIEVAL | VALEUR1    | VALEUR2    | SCORE_JARO_WINKLER |
|----|--------------|------------|------------|--------------------|
| 1  | CITY         | Londres    | Londre     | 97                 |
| 2  | CITY         | Paris      | Pari       | 96                 |
| 3  | CITY         | Paris      | PArisss    | 94                 |
| 4  | FIRSTNAME    | Adam       | ADAM       | 100                |
| 5  | FIRSTNAME    | Adam       | Adams      | 96                 |
| 6  | FIRSTNAME    | Marie-Noel | Marie Noel | 96                 |
| 7  | FIRSTNAME    | Rahma      | Rama       | 94                 |
| 8  | FIRSTNAME    | Mbarak     | Moubarak   | 92                 |
| 9  | FIRSTNAME    | Franc      | Frank      | 92                 |
| 10 | FIRSTNAME    | Adam       | Adem       | 86                 |

|     |          |                                                                   |
|-----|----------|-------------------------------------------------------------------|
| 255 | SELECT   | categorieval, valeur1, valeur2,                                   |
| 256 |          | q_gram_similarity(UPPER(valeur1), UPPER(valeur2)) AS Score_Q_Gram |
| 257 | FROM     | matchval                                                          |
| 258 | WHERE    | categorieval = 'FIRSTLASTNAME'                                    |
| 259 |          | AND q_gram_similarity(UPPER(valeur1), UPPER(valeur2)) > 75        |
| 260 | ORDER BY | Score_Q_Gram DESC;                                                |
| 261 |          |                                                                   |

  

|          |        |               |          |              |              |               |             |
|----------|--------|---------------|----------|--------------|--------------|---------------|-------------|
| Problems | Output | Debug Console | Terminal | Query Result | Task Monitor | Script Output | SQL History |
|----------|--------|---------------|----------|--------------|--------------|---------------|-------------|

  

All rows fetched: 6 in 0.072 seconds

|   | CATEGORIEVAL  | VALEUR1        | VALEUR2        | SCORE_Q_GRAM |
|---|---------------|----------------|----------------|--------------|
| 1 | FIRSTLASTNAME | Superman       | Superman       | 100          |
| 2 | FIRSTLASTNAME | Peter Parker   | peter parker   | 100          |
| 3 | FIRSTLASTNAME | Brigitte Bardo | Brigitte Fardo | 85           |
| 4 | FIRSTLASTNAME | Wonder Woman   | Ponder Woman   | 82           |
| 5 | FIRSTLASTNAME | Harissa FORD   | Harisson Ford  | 78           |
| 6 | FIRSTLASTNAME | Peter Parker   | Pete Parker    | 76           |

#### 4.4 Règle de décision MATCH

Après normalisation (majuscules, suppression des accents, des espaces et de la ponctuation), nous calculons pour chaque colonne texte un score = max(EDS, JWS, Q-gram), puis nous appliquons les règles suivantes. Les seuils sont des propositions de départ, à calibrer sur les données :

- **Identique après normalisation.** Doublet exact.
- **Score  $\geq 85$ , au moins un attribut de confirmation égal (date de naissance, email, téléphone) et aucun conflit.** Doublet.
- **Score  $\geq 85$  mais un conflit (date ou civilité différente).** Distinct.
- **Score entre 60 et 85, ou aucun attribut de confirmation disponible.** À vérifier : aucune fusion automatique.
- **Score  $< 60$ .** Distinct, sauf correspondance dans une table de référence ou proximité sémantique (traductions).

Soundex et Q-gram servent d'indices complémentaires et ne décident jamais seuls. Cette règle remplace le test actuel de la procédure ELIMINEDOUBSIMIL (EDS > seuil et JWS > seuil et 3 premiers caractères identiques), qui force un choix binaire sur une clé concaténée.

#### 4.5 Stratégie de fusion MERGE

Dans le cadre de l'amélioration de la qualité des données, la détection des doublons (par exemple via les algorithmes de Jaro-Winkler ou les Q-Grams) n'est qu'une étape préliminaire. Lorsqu'un algorithme identifie que deux lignes décrivent en réalité la même entité (un même client, par exemple), la suppression arbitraire de l'une des deux lignes entraînerait une perte d'information.

Pour y parvenir, notre stratégie s'appuie sur les métadonnées de profilage stockées dans la table dictionnaire **META02\_DD\_MESURES**. Les indicateurs statistiques de cette table agissent comme des "arbitres" pour décider quelle valeur conserver en cas de conflit entre deux enregistrements similaires.

Voici les règles d'arbitrage que nous avons définies :

**1. Complétion mutuelle (Gestion des valeurs manquantes) :** La règle la plus fondamentale de la fusion est l'enrichissement de l'enregistrement. Si deux lignes sont similaires, mais que la ligne A possède une

valeur manquante (NULL) dans une colonne où la ligne B contient une information valide, l'enregistrement maître récupère la valeur de la ligne B. Si les deux lignes entrent en conflit sur la complétude globale, nous utilisons les métriques M100 (Number of MISSING values) et M101 (Number of NON-MISSING values) pour désigner la ligne source la plus riche (celle ayant le moins de valeurs nulles) comme base prioritaire de la fusion.

- Exemple : La ligne A contient le téléphone 0612345678 mais aucune ville. La ligne B contient la ville PARIS mais aucun téléphone. L'enregistrement maître fusionné contiendra à la fois PARIS et 0612345678.

**2. Arbitrage par la richesse de l'information (Longueur des chaînes) :** Lorsque deux lignes entrent en conflit sur une même colonne avec des données valides mais différentes (par exemple, des abréviations), l'algorithme privilégie la donnée la plus détaillée. Pour cela, nous interrogeons la métrique M109 (MAXIMUM length of strings) du dictionnaire de mesures.

- Exemple : La ligne A indique le prénom C. et la ligne B indique Clémence. Les deux sont jugées très similaires par les algorithmes, mais au moment de la fusion, la règle basée sur la métrique M109 sélectionnera automatiquement Clémence, considérée comme la donnée la plus complète et la plus exploitable.

**3. Conformité au type syntaxique dominant :** Dans certains cas, une erreur de saisie provoque un conflit où l'une des valeurs est une aberration syntaxique. La table de mesures nous fournit la métrique M120 (DOMINANT SYNTAX TYPE). Lors de la fusion, la valeur conservée sera celle qui respecte le type syntaxique majoritaire de la colonne.

- Exemple : Si la ligne A indique un code postal 93800 et la ligne B un code postal Paris (erreur de décalage de colonne), la confrontation avec le type dominant de la colonne CPCLI (qui est strictement numérique) permettra d'éliminer automatiquement la chaîne de caractères et de conserver 93800 pour l'enregistrement final.

**4. Actualisation par les données temporelles :** Pour les données évolutives ou les dates, la fusion doit privilégier l'information la plus fraîche. En nous appuyant sur la métrique M167 (Maximum value of dates), l'algorithme est capable de résoudre un conflit de dates en conservant systématiquement la date la plus récente lors de la consolidation.

- Exemple : En cas de doublon où la ligne A possède une date de dernier contact au 12-12-2012 et la ligne B au 20-10-2020, l'enregistrement consolidé adoptera automatiquement le 20-10-2020.

Cette approche dirigée par les métadonnées de la table META02\_DD\_MESURES transforme la simple élimination des doublons en un véritable processus curatif. En sélectionnant la meilleure information possible pour chaque attribut en fonction de règles statistiques claires, la base de données résultante devient structurellement plus riche et plus fiable que les sources de départ.

N.B. : On peut aussi choisir des colonnes selon :

- La priorité à la valeur validée par une table de référence, sinon la plus fréquente dans le groupe, sinon la plus récente ou celle de la source la plus fiable.

- On conserve la forme canonique : accents conservés, majuscule initiale, téléphone au format +33.
- Une table de correspondance ancien identifiant → identifiant survivant est conservée, et la table source S reste intacte.

#### 4.6 Piste : utiliser un embedding

Un embedding transforme une chaîne en vecteur, et la similarité cosinus mesure la proximité de sens. Il apporterait notamment les traductions (Pékin / Beijing, Londres / London). Il présente aussi un risque de faux positifs, car il mesure la proximité de sens et non l'identité (The Hulk / Iron Man). Cette piste n'a pas été exécutée à ce stade : elle suppose soit un calcul externe (Python, modèle multilingue) avec rechargement des scores dans Oracle, soit une version d'Oracle gérant le type VECTOR.

## 5. Conclusion

En conclusion, ce travail pratique nous a permis d'explorer en profondeur les problématiques complexes liées à la qualité et à l'intégration des données au sein d'une base de données relationnelle sous Oracle.

Nous avons pu mettre en œuvre et évaluer l'efficacité de différents algorithmes de calcul de similarité (comme l'Edit Distance, Jaro-Winkler, les Q-grams et les codes phonétiques Soundex), tout en comprenant l'importance cruciale d'adapter le choix de l'algorithme et des seuils à la sémantique de chaque colonne. La mise en place d'une stratégie de détection rigoureuse (MATCH), combinée à une consolidation intelligente attribut par attribut (MERGE) guidée par les métadonnées de profilage (META02\_DD\_MESURES), a démontré qu'il est possible de transformer un simple nettoyage de doublons en un véritable processus d'enrichissement et de fiabilisation de l'information.

Enfin, l'étude de pistes avancées, telles que l'utilisation d'intégrations vectorielles (Embeddings), ouvre des perspectives prometteuses pour surmonter les limites des approches lexicales strictes. Ce projet souligne ainsi qu'une gestion moderne et performante de la qualité des données repose sur une synergie indispensable entre règles syntaxiques, contraintes sémantiques et automatisation intelligente.